Dusk Park — Design Document

Live demo: `https://g36maid.github.io/cg-final-project/Theme-Park/`

Student ID: 41173058H
Name: 鍾詠傑 (Chung Yung-Chieh)

Figure 1: The Dusk Park hub at golden hour. The player stands in the central plaza with the reflecting fountain (T6), procedurally textured buildings, warm lamp point lights (T3), the dusk-gradient skybox (T5), and long dramatic shadows cast by the directional sun (T7).

Contents

1 Game Concept

Dusk Park is a 3D theme-park meta-game. The player wanders an outdoor plaza at dusk and visits four integrated sub-games: a neon *Pinball* table, a *Rubik's Cube* solver, a *3D Tetris* well, and a *Roller Coaster* ride. Three of these sub-games (Pinball, Rubik's Cube, Tetris) are arcade machines that *award* tokens on completion; the Roller Coaster *consumes* tokens per ride. The meta-objective is simple and clearly communicated to the player through the HUD and an information board:

> *Earn tokens by playing arcade games, then spend them on coaster rides. After three rides the tour is "complete" and a fireworks celebration plays.*

The hub itself is a first-person walkthrough scene that concentrates every hard graphics requirement of the course spec (T1–T7): player movement, first/third-person camera, Phong point lights, textured surfaces, a skybox cubemap, cubemap reflection on the fountain water, and shadow mapping. The sub-games are integrated through page navigation and `localStorage`-based token persistence, so each sub-game keeps its own renderer and gameplay intact.

The experience we want the player to have is one of *wandering a park caught between day and night*. Dusk is the narrative metaphor: the park sits at the boundary of reality (the hub) and the game universes behind each door, and the warm light makes every graphics technique read clearly on screen.

2 Scene & Visual Design

2.1 Why Dusk?

We deliberately rejected both daytime and night. Under a daytime sky the point-light contribution is visually negligible (sun washes it out) and shadows are short and flat; at night the coaster landscape is invisible and the arcade exteriors lose their context. **Dusk** is the single time of day where every required technique is simultaneously visible and dramatic:

- **Point lights + Phong** — the low ambient lets the warm lamp lights and their ambient/diffuse/specular components all read on the dark building facades.
- **Shadows** — the low sun angle stretches building shadows across the plaza, making the shadow map obvious rather than subtle.
- **Skybox** — the orange-to-purple dusk gradient is visually richer than a flat blue day sky and is cheap to generate procedurally.

- **Reflection** — the fountain water reflects the colourful sky and neon signs, which is far more legible than reflecting a uniform blue daytime sky.

2.2 Colour Palette

The palette (Table 1) is built around a warm-cool tension: warm orange/gold for the sun and lamps, cool purple/indigo for the sky and ambient, and saturated neon pink/cyan as accents that bridge the hub to the arcade interiors.

Table 1: Key colours and their roles.

Role	Hex	Purpose
Sky zenith	`#0A0418`	Deep indigo, anchors the top of the cubemap
Sky horizon	`#FF7A35`	Warm sunset band where the eye is drawn
Ambient	`#1F1A2E`	Cool purple fill, matches the sky's cool side
Sun (fill)	`#FFA65E`	Low-angle directional light at intensity 0.35
Lamp (point)	`#FFD98C`	Warm yellow point lights at intensity 1.8
Neon pink	`#FF4D99`	Arcade hall accent, banners
Neon cyan	`#4DDEF0`	Signage borders, fountain sparkles
Stone ground	`#59504A`	Neutral absorptive surface; takes shadows well

2.3 Objects in the Scene

The plaza contains three landmark buildings, a central fountain, an information board, street lamps, benches, planters, and decorative banners. Each major object earns its place: the **Arcade Hall** (neon-striped facade) is the portal to the three arcade sub-games; the **Coaster Station** (metal grid facade) gates the token-gated ride; the **Tour Train** station (wooden planks) anchors the far end of the plaza. The **central fountain** is the showcase for cubemap reflection (T6) and sits at the visual centre so the player encounters it immediately. The **information board** at the entrance provides the in-game instructions (D6).

3 Technical Choices

3.1 Camera & Projection

The camera uses a perspective projection with a **65° FOV**. We chose a slightly wider-than-default FOV because the park is an open plaza the player must take in laterally; a narrow FOV would feel tunnel-visioned when standing among the buildings. The **near plane is 0.1** and the **far plane is 1000**; the large far plane is required because the skybox box is 500^3 units and must never be clipped. The canvas fills the browser viewport (16:9 in the screenshot), so the aspect ratio matches the display.

In first person the camera sits at the player's **eye height of 1.7 units**, a deliberately human scale so the buildings (7–9 units tall) tower over the player and the fountain (2.3-unit orb) is at a comfortable viewing angle. Pressing `C` switches to a third-person view that pulls the camera 6 units back and 3 units up; this exists to satisfy the spec's "meaningful camera control" (T2) and to let the player see their own context and the long shadows they cast.

Mouse look uses pointer lock with a sensitivity of **0.0022 rad/px**; pitch is clamped to ±89° to avoid gimbal flip. Movement is WASD with an exponentially-damped velocity (`blend = 1 - exp(-18 * dt)`) rather than instant snap, so the camera glides to a stop—this hides the aliasing of discrete key presses and prevents the scene from feeling robotic.

3.2 Lighting

3.2.1 Directional Sun (Fill)

The directional sun is intentionally a *fill* light, not the hero. It uses a warm orange colour (`[1.0, 0.65, 0.38]`) at a modest **intensity of 0.35** with direction `[-0.4, 0.35, -0.7]`. We kept the intensity low because the real visual work is done by the point lights: a bright sun would flatten the contrast between lit lamp halos and the unlit ground. The direction places the sun low and to the left-rear, which casts building shadows *forward and right across the plaza* where the player walks, making the shadow-mapping technique (T7) immediately visible from the spawn point. Had the sun been on the other side, the shadows would fall away from the plaza and the technique would be invisible.

3.2.2 Point Lights

There are **four warm lamp point lights** (colour `[1.0, 0.85, 0.55]`, intensity **1.8**) placed on the plaza corners at height 5, plus a cooler **fountain accent light** (colour `[0.45, 0.75, 1.0]`, intensity 0.8) at the centre. The Phong shader supports up to 8 point lights via a uniform array. Four corner lamps were chosen (rather than two or one) because the plaza is 40+ units across: with fewer lights the far buildings would be unlit and the specular term—the whole point of the Phong requirement—would never fire on them. The attenuation model is $\frac{1}{1+0.05d+0.005d^2}$; the quadratic term kicks in around $d \approx 20$ which is roughly the plaza radius, so lamps brighten their immediate neighbourhood without washing out the scene globally.

The point lights are *not* shadowed (only the directional sun is). Shadowing 4 point lights would require 4 additional depth passes (or a cubemap shadow per light), which is prohibitively expensive for a browser game. This is a deliberate cost-quality trade-off.

3.2.3 Ambient

The ambient term is a cool purple `[0.12, 0.10, 0.18]`, deliberately tinted toward the sky's cool side rather than grey. A neutral grey ambient on a dusk scene reads as "wrong" because the eye expects skylight to be blue-ish; the purple tint sells the time of day. The intensity is low (0.12) so that the unlit sides of buildings still read as "in shadow" and the lit sides pop.

3.3 Phong Materials

The shader computes, per fragment: `colour` $= k_a \cdot \text{base} \cdot \text{ambient} + \sum_i (k_d \cdot \text{NdotL} + k_s \cdot \text{spec}) \cdot \text{light}_i$. Each major surface has a hand-tuned material (Table 2). The reasoning for each follows.

Table 2: Phong material parameters per surface. Values are linear-RGB coefficients.

Surface	k_a	k_d	k_s	**shin.**	**Rationale**
Stone ground	0.15	0.35	0.15	8	Rough, absorptive; low diffuse so lamps must light it; very low shininess for a matte look
Buildings	0.20	0.55	0.30	32	Painted concrete; moderate specular so lamp halos glance off walls; shininess 32 gives a medium-tight highlight
Metal (lamps, station)	0.10	0.30	0.90	128	High specular (0.90) and high shininess (128) so lamps produce a tight bright glint—this is what "metal" should look like under a point light; low diffuse so the metal reads as reflective rather than painted
Fountain orb	0.10	0.30	0.90	128	Same metal preset as the lamps, tinted blue, to read as a polished sphere catching the cool fountain light
Arcade facade	0.14	0.42	0.55	48	Slightly higher specular than generic buildings so the neon texture's bright bands catch the lamps

Per-parameter justification for the key surfaces:

- **Ground** (*the surface the player walks on*): k_a=0.15 is just enough to keep the ground visible in shadow; k_d=0.35 is deliberately *low* so that the ground does not self-light and compete with the lamps—the ground should *receive* light, not generate the impression of light. k_s=0.15 with shininess 8 gives a broad, dull sheen appropriate for weathered stone; a high shininess here would produce unrealistic tiny hotspots on a rough material.
- **Metal** (*lamp heads, coaster station, fountain orb*): k_s=0.90 is the highest in the scene because metal's defining visual property is that it *reflects direct light sharply toward the viewer*. The shininess of 128 makes the specular highlight a tight bright dot—exactly how a polished metal sphere reads under a point light. We did *not* raise k_d to match; keeping diffuse at 0.30 ensures the metal is dark between highlights, which is what makes the highlights read as reflections rather than as the surface's own colour.
- **Buildings**: the diffuse 0.55 is higher than the ground because buildings are the primary surfaces the player looks at and should carry their texture colour well; the specular 0.30 at shininess 32 is the sweet spot where a lamp halo is visible on the wall without the wall looking metallic.

3.4 Textures

3.4.1 Procedural Stone Ground (T4)

The plaza floor uses a 256×256 procedural texture generated on a `<canvas>`: a per-pixel noise grain over a warm stone base, 32-px tile grout lines, and 18 hand-drawn crack polylines. The UVs are tiled so the texture repeats ≈ 33 times across the 200×200 ground plane (`wrapS/T = REPEAT`, mipmaps on). We chose procedural over a photo texture because (a) the monorepo carries no image assets and (b) procedural tiling avoids the visible seam a single photo would produce. The grout lines give a sense of scale; without them a flat noise would read as "infinite gravel" rather than "paved plaza".

3.4.2 Building Facades

Each building has a distinct procedural texture: the Arcade Hall gets neon pink/cyan horizontal bands over a dark noisy base (echoing the neon pinball interior); the Coaster Station gets a brushed-metal diagonal gradient with an orange bolt grid; the Tour Train gets wooden planks with bezier grain. These are not random—they telegraph the sub-game behind each door before the player reads the sign.

3.4.3 Skybox (T5)

The skybox is a 500^3 unit box with a procedurally generated cubemap. Each side face is a vertical gradient: indigo zenith → purple band → orange-red → bright orange sunset → gold → dark ground. The $+Y$ (top) face is pure indigo with 120 randomly placed stars; the $-Y$ (bottom) face is flat warm dark. The box uses `cullFace = GL_FRONT` (so we see the inside), `depthWrite = false` (so it does not occlude geometry), and `frustumCulled = false` (so OGL does not cull it when the camera is off-origin).

3.5 Shadow Mapping (T7)

Shadows are produced by OGL's `Shadow` extra: an orthographic "sun camera" matched to the directional light, rendering a 2048×2048 **depth map**. The ortho frustum is ± 30 units with near 0.5 and far 80, positioned 40 units up the sun vector looking at the origin. The Phong fragment shader projects each fragment into the sun camera's clip space, samples the depth map, and applies a shadow factor of **0.35** (i.e. shadowed regions keep 35% of the sun's contribution, not 0%) with a depth bias of **0.002**. We chose 0.35 rather than full black because a dusk scene's shadows are soft-filled by skylight; pitch-black shadows would look like high noon, not dusk. The bias of 0.002 suppresses acne without peter-panning at this scale.

3.6 Cubemap Reflection on the Fountain (T6)

The fountain water surface is a thin cylinder rendered with a custom shader that reflects the skybox cubemap. For each fragment it computes the view direction $\mathbf{I}$, the surface normal $\mathbf{N}$, the reflection vector $\mathbf{R} = \text{reflect}(\mathbf{I}, \mathbf{N})$, and samples $\text{textureCube}(tEnvMap, \mathbf{R})$. A Fresnel term $F = (1 - \max(-\mathbf{I} \cdot \mathbf{N}, 0))^3$ modulates the mix: at grazing angles (where the player looks across the water) reflection dominates (strength 0.9); looking straight down, the water's own dark teal colour (strength 0.45) shows through. This matches the physical behaviour of water—reflective at shallow angles, transparent when viewed from above—and makes the dusk sky "paint" itself onto the fountain.

3.7 Token Economy & Game Mechanics (D1–D6)

All six mechanics dimensions are implemented: **D1** clear objective (earn → ride ×3 → celebration); **D2** collision/interaction (AABB colliders on every building, lamp, bench, planter; E-key proximity triggers at each door); **D3** scoring visible (HUD shows token count and coaster rides); **D4** win/lose (soft win: 3 rides triggers a fireworks celebration overlay; no lose state, by design); **D5** feedback (token-gain/loss toasts, proximity prompts, celebration animation); **D6** instructions (entrance information board + M-key help panel listing objective, controls, and facility roles).

4 Integrated Sub-Games

The four sub-games are integrated via page navigation (`location.href`) with a `?from=hub` URL parameter. Token state persists in `localStorage['dusk-park-state']`. A shared hooks mod-

ule (`Theme-Park/src/meta/hooks.js`) injects a "return to park" button and handles payout events, so sub-game physics and rendering are untouched.

Figure 2: 3D Pinball: hand-rolled 2D physics on a tilted table, a PBR metal ball with cubemap reflection, Bloom post-processing, and Web Audio SFX. Awards 10 tokens on game over.

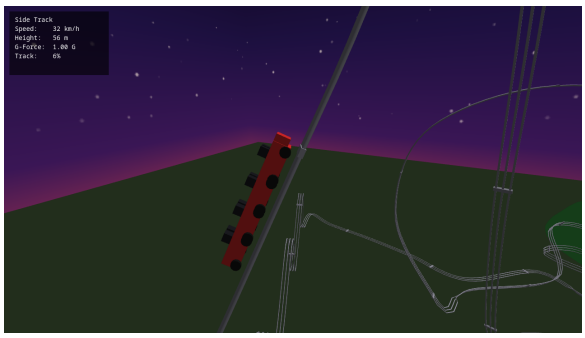

Figure 3: Roller Coaster: Catmull-Rom spline track with Frenet–Serret (TNB) frame orientation, energy-conservation physics, and a 2D-canvas HUD (speed / height / G-force / progress). Consumes 10 tokens per ride.

Figure 4: Rubik's Cube: raycast face-picking, drag-to-rotate with `easeOutBack` snap, a layer-by-layer auto-solver, and a live 2D cross-net. Awards 10 tokens on solve.

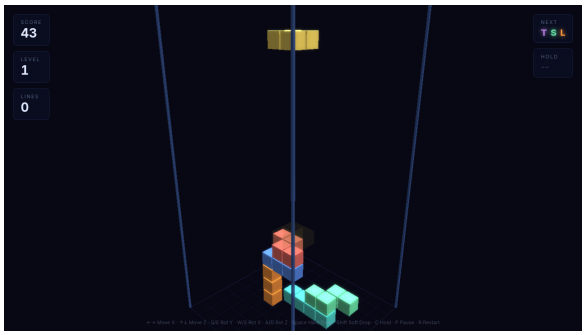

Figure 5: 3D Tetris: a $10\times20\times10$ volumetric well with SDF round-edge blocks, three-axis rotation (wall-kick corrected), ghost-piece preview, Bloom post-processing, and a grey-scale game-over fade. Awards 10 tokens on game over.

5 Challenges and Solutions

5.1 The Skybox Was Invisible: the `mat3 + xyww` Trap

Symptom. The skybox mesh (500^3 box, cubemap texture) was completely invisible. Changing the fragment shader to output pure red produced no red pixels. No WebGL errors.

Root cause. The textbook skybox trick uses `gl_Position = (projection * mat3(modelView) * pos).xyww` to force $z = w$ so the box sits on the far plane. The `mat3()` strips translation; `xyww` forces depth. This assumes the camera is at the box centre. In our walkthrough scene the camera is at `[0, 1.7, 18]`, *not* at the origin. For vertices behind the camera (e.g. the $+Z$ face at view-space $z = 250$), the post-projection w is negative, and the `xyww` trick sets $z = w < 0$. The clip test $-w \le z \le w$ then becomes "positive $\le$ negative $\le$ negative", which fails. Roughly half the box's vertices were clipped away, collapsing most triangles.

Fix. Abandon the `mat3/xyww` trick entirely. Use standard projection (`gl_Position = projection`

`* modelView * vec4(pos,1)`) with a physically giant box (500^3) centred at the world origin so the camera is always inside it. Render with `cullFace = GL_FRONT`, `depthWrite = false`, and `frustumCulled = false`. This is robust for any camera position.

5.2 OGL Array-Uniform Naming

Symptom. With `uniform vec3 uLightPos[8]` in the shader, OGL logged "Active uniform uLightPos[0] has not been supplied" every frame, even though we passed values.

Root cause. OGL's `Program` parses the active-uniform name `uLightPos[0]` with a regex and looks up `this.uniforms['uLightPos']`. If the uniforms object was keyed as `'uLightPos[0]'` (as a naive port from raw WebGL would do), the lookup misses. Separately, OGL's `flatten()` checks `Array.isArray(value)`; a `Float32Array` returns `false` and is misinterpreted as a struct.

Fix. (1) Key uniforms by the base name only (`uLightPos`, not `uLightPos[0]`). (2) Pass a plain JavaScript `Array` (not `Float32Array`) as the value. OGL then flattens and uploads via `uniform3fv` correctly.

5.3 ES-Module Import Paths and the `miniserve` Symlink

Symptom. `import {…} from '../../ogl/src/index.js'` 404'd when the server root was the `Theme-Park/` directory, because the browser normalised the relative path back to the server root where `ogl/` did not exist.

Fix. Serve from the *repo root* (so all sibling game directories resolve), and add a `Theme-Park/ogl -> ../ogl` symlink so that the `../../ogl/...` path also resolves when `Theme-Park/` is served alone. `miniserve` must not be passed `--no-symlinks`.

6 Most Proud Of

What we are most proud of is not any single technique but the **project's trajectory**. The four sub-games did not start as components of a theme park—they began as independent experiments. While learning the OGL library we built each game in isolation: a neon Pinball table to push hand-rolled physics and PBR metal, a Rubik's Cube to solve quaternion layer-rotation and raycast face-picking, a 3D Tetris to exercise SDF shaders and Bloom post-processing, and a Roller Coaster to tackle Catmull-Rom splines and Frenet–Serret framing. Each was a self-contained playground that explored a different corner of real-time computer graphics.

The insight that turned four demos into one game was the **theme-park hub**. Instead of discarding the prototypes or rewriting them into a single renderer (an engineering nightmare with four independent OGL scenes), we asked: what if the park itself *is* the integration layer? A first-person plaza where each door leads to a different game universe, connected by a shared token economy and a persistent save state. The hub then becomes the single place where every hard graphics requirement—skybox, Phong point lights, shadow mapping, cubemap reflection, textured surfaces, camera control—is concentrated and shown off at dusk, the one lighting condition where they all read simultaneously.

So the real achievement is the *architecture*: the four games kept their original physics, rendering, and gameplay untouched, and the hub glued them together with nothing more than page navigation, `localStorage`, and a shared hooks module. What was originally "four ways to explore a library" became *Dusk Park*—one cohesive game where the player earns tokens in an arcade, spends them on a coaster ride, and watches fireworks after three tours, all set against a dusk sky that makes every graphics technique visible in a single frame.